Algoritma dan Pemrograman Komputer 1

Pertemuan 6: Pernyataan Percabangan

M. Ma'ruf Qomaruddin Kafi – 5002241095
Thalita Ayu Antoniasari – 5002251011

05 November, 2025

Departemen Matematika
Fakultas Sains dan Analitika Data
Institut Teknologi Sepuluh Nopember

Daftar Isi

Pengenalan Pernyataan Percabangan

BREAK Statement

CONTINUE Statement

RETURN Statement

Perbandingan BREAK vs CONTINUE vs RETURN

Best Practices

Contoh Kasus

Latihan

Kesimpulan

Referensi

Pengenalan Pernyataan Percabangan

Konsep Pernyataan Percabangan dalam Java

Apa itu Percabangan?

Pernyataan percabangan adalah pernyataan yang digunakan untuk **mengatur alur eksekusi** program dengan cara mengubah urutan normal eksekusi kode.

Catatan istilah: Di Java, `break`, `continue`, dan `return` lebih tepat disebut *branching/jump statements*. Istilah *percabangan/selection* biasanya dipakai untuk `if`, `if-else`, dan `switch`.

Seperti **tombol kontrol** dalam program untuk:

- **Berhenti** di tengah proses (`break`)
- **Lompati** langkah tertentu (`continue`)
- **Keluar** dari fungsi (`return`)

Analoginya:
Bayangkan sedang membaca buku:

- `break` = berhenti baca bab ini
- `continue` = lewati halaman ini
- `return` = tutup buku, selesai membaca

BREAK Statement

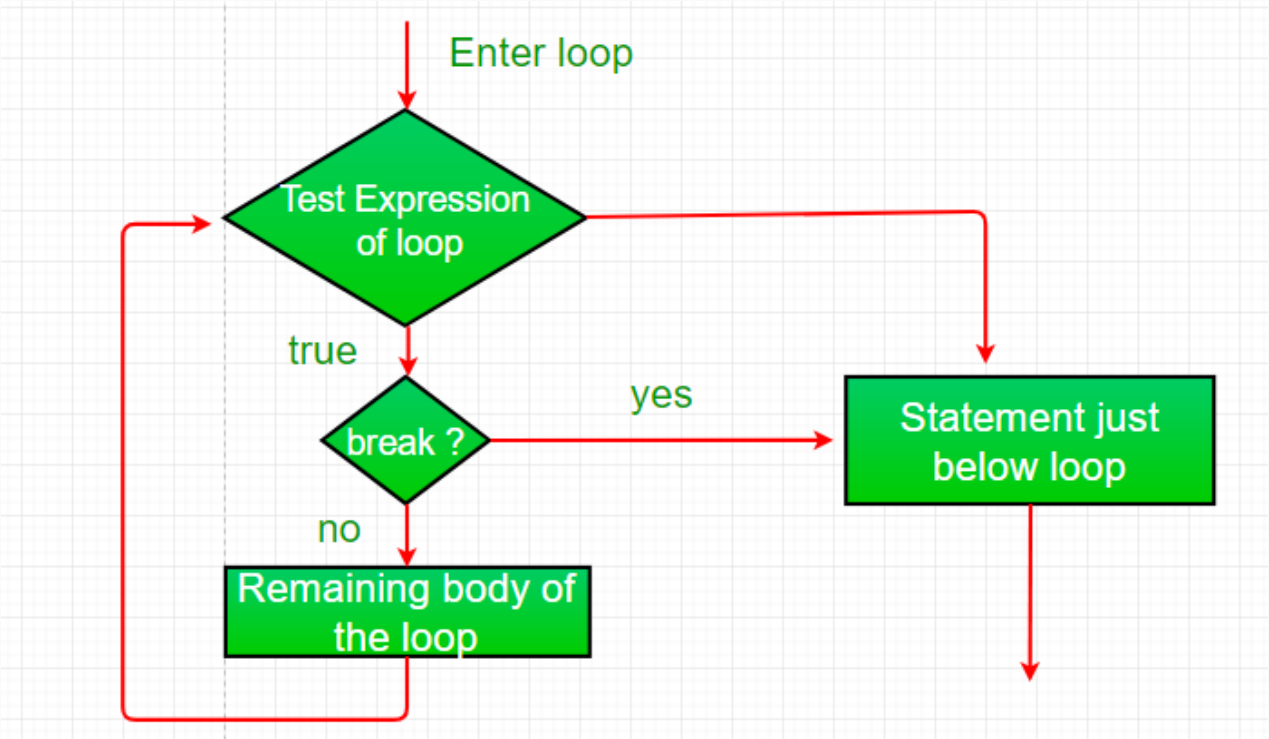
Sintaks dan Konsep BREAK

Fungsi BREAK Statement
Menghentikan eksekusi perulangan (for, while, dan do-while) atau percabangan (switch case) secara paksa dan keluar dari blok kode tersebut.

Jenis BREAK	Deskripsi
`break` (tanpa label/unlabeled)	Menghentikan perulangan/percabangan terdekat
`break label` (dengan label/labeled)	Menghentikan perulangan/percabangan dengan label tertentu

Tabel 1: Jenis-jenis BREAK Statement

Flowchart BREAK



Gambar 1: Flowchart Break

Sumber: GeeksforGeeks - Java Break Statement

BREAK (Tidak Berlabel/Unlabeled)

Contoh BREAK dalam FOR Loop

```
1  public class BreakUnlabeled {
2      public static void main(String[] args) {
3          for (int i = 0; i < 5; i++) {
4              System.out.println("i = " + i);
5              if (i == 2) {
6                  break;  // Keluar loop saat i = 2
7              }
8          }
9          System.out.println("Loop selesai");
10     }
11 }
12
```

Output Program

```
i = 0

i = 1

i = 2

Loop selesai
```

BREAK (Dengan Label/Labeled)

Contoh BREAK dengan Label dalam WHILE Loop

```
1  public class BreakLabeled {
2      public static void main(String[] args) {
3          int x = 35;
4          int y = 4;
5          int dummy = y;
6
7          mulai:
8          while(true) {
9              if(y > x) {
10                 System.out.println("Nilai kelipatan y yang mendekati x adalah " + y);
11                 break mulai;  // Keluar dari blok 'mulai'
12             }
13             y = y + dummy;
14         }
15     }
16 }
17
```

Output Program

```
Nilai kelipatan y yang mendekati x adalah 36
```

Perbandingan BREAK Berlabel vs Tidak Berlabel

Konteks Pemakaian	`break;`	`break label;`
Perulangan / Loop (`for/while/do`)	✓	✓
Label pada Loop (`label: for...`)	✓	✓
`if else` (di dalam loop)	✓	✓
`if else` (di luar loop)	×	×
`switch`	✓	✓
Label pada Blok (`label: {...}`)	×	✓

Tabel 2: Perbandingan konteks pemakaian `break` berlabel vs tidak berlabel

CONTINUE Statement

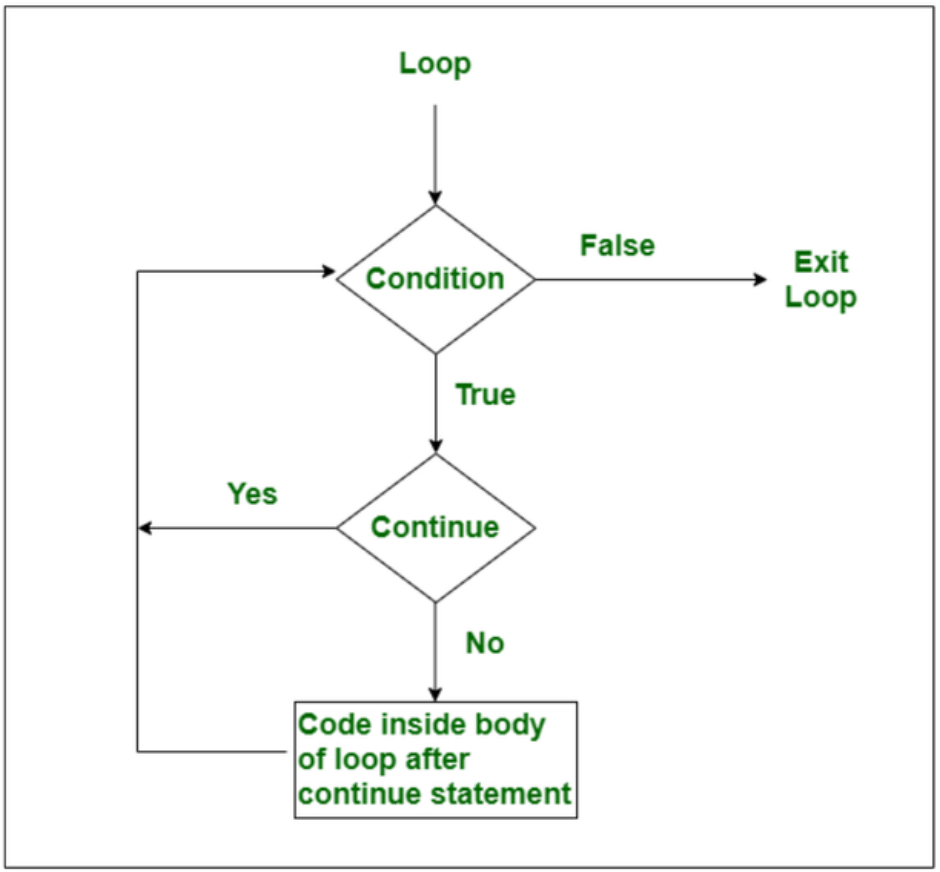
Sintaks dan Konsep CONTINUE

Fungsi CONTINUE Statement
Melompati iterasi saat ini dalam perulangan dan langsung melanjutkan ke iterasi berikutnya.

Jenis CONTINUE	Deskripsi
`continue` (tanpa label/unlabeled)	Melompati iterasi dalam perulangan terdekat
`continue label` (dengan label/labeled)	Melompati iterasi dalam perulangan berlabel

Tabel 3: Jenis-jenis CONTINUE Statement

Flowchart Continue



Gambar 2: Flowchart Continue

Sumber: GeeksforGeeks - Java Continue Statement

CONTINUE Tidak Berlabel (Unlabeled)

Contoh CONTINUE dalam FOR Loop

```
1  public class ContinueUnlabeled {
2      public static void main(String[] args) {
3          for(int i = 0; i < 5; i++) {
4              if(i == 2) {
5                  continue;  // Lewati saat i = 2
6              }
7              System.out.println("i = " + i);
8          }
9      }
10 }
11
```

Output Program

```
i = 0

i = 1

i = 3

i = 4
```

CONTINUE dengan Label (Labeled)

Contoh CONTINUE Berlabel dalam Nested Loop

```
1  public class ContinueLabeled {
2      public static void main(String[] args) {
3          minggu:
4          for (int i = 1; i <= 3; i++) {
5              for (int j = 1; j <= 3; j++) {
6                  if (j == 2) {
7                      continue minggu; // Skip sisa hari, lanjut ke minggu depan
8                  }
9                  System.out.println("Minggu " + i + ", Hari " + j);
10             }
11         }
12     }
13 }
14
```

Output Program

```
Minggu 1, Hari 1

Minggu 2, Hari 1

Minggu 3, Hari 1
```

Penjelasan:

- Karena `j == 2`, program langsung `continue minggu;`.
- **Hari 2 dan Hari 3 dilewati**, dan program langsung loncat ke **Minggu berikutnya**.

Perbandingan CONTINUE Berlabel vs Tidak Berlabel

Konteks Pemakaian	`continue;`	`continue label;`
Perulangan / Loop (`for/while/do`)	✓	✓
Label pada Loop (`label: for...`)	✓	✓
`if else` (di dalam loop)	✓	✓
`if else` (di luar loop)	×	×
`switch` (di luar loop)	×	×
`switch` (di dalam loop)	✓	✓
Label pada Blok (`label: {...}`)	×	×

Tabel 4: Perbandingan konteks pemakaian `continue` berlabel vs tidak berlabel

RETURN Statement

Fungsi RETURN Statement

Menghentikan eksekusi program atau method. Dalam **main method** return digunakan untuk menghentikan program secara paksa.

Catatan: Pembahasan lengkap tentang method dan return akan dibahas di Modul 9 (Fungsi/Method). Di slide berikutnya hanya diberikan **preview singkat**.

Jenis RETURN	Deskripsi
`return` (di main)	Menghentikan program secara paksa
`return value` (di method)	Mengembalikan nilai

Tabel 5: Jenis-jenis RETURN Statement

RETURN untuk Menghentikan Program

Contoh RETURN di Main Method

```java
import java.util.Scanner;

class kembali {
    public static void main (String args[]) {
        System.out.println("Masukan nilai suatu bilangan: ");
        Scanner baca = new Scanner(System.in);
        int k = baca.nextInt();

        if(k % 2 == 0) {
            System.out.println(k + " merupakan bilangan genap");
            return;  // BERHENTI di sini ketika k genap
        }

        System.out.println(k + " merupakan bilangan ganjil");
    }
}

```

Output Program (input = 4)

```
Masukan nilai suatu bilangan:
4
4 merupakan bilangan genap
```

RETURN untuk Menghentikan Program (Part 2)

Method check() dengan main method

```java
import java.util.Scanner;

public class CekBilangan {
    public static String check(int k) {
        if (k % 2 == 0) {
            return "genap";
        } else {
            return "ganjil";
        }
    }

    public static void main(String[] args) {
        System.out.println("Masukan nilai suatu bilangan:");
        Scanner baca = new Scanner(System.in);
        int k = baca.nextInt();

        System.out.println(k + " merupakan bilangan " + check(k));
    }
}

```

Output Program (jika input = 7)

```
Masukan nilai suatu bilangan:

7

7 merupakan bilangan ganjil
```

Perbandingan BREAK vs CONTINUE vs RETURN

Perbandingan 5 Statement Percabangan

Kriteria	`break;`	`break label;`	`continue;`	`continue label;`	`return;`
Efek	Hentikan blok terdekat	Hentikan blok berlabel	Skip ke iterasi berikutnya	Skip ke iterasi blok berlabel	Keluar dari method
Scope	Loop/switch terdekat	Loop berlabel	Loop terdekat	Loop berlabel	Method
Eksekusi selanjutnya	Setelah blok loop	Setelah blok berlabel	Iterasi berikutnya	Iterasi loop berlabel	Kode pemanggil
Nested loop	Keluar 1 level	Keluar banyak level	Skip 1 iterasi	Skip ke outer loop	Keluar semua level
Bisa di switch?	✓	✓	!	!	✓
Dapat berlabel?	×	✓	×	✓	×

Tabel 6: Perbandingan 5 varian statement percabangan

! = `continue`/`continue label` hanya legal jika `switch` berada di dalam loop.

Best Practices

Best Practices Penggunaan Percabangan

Tips dan Rekomendasi

- **Gunakan BREAK dengan hati-hati** - Dapat membuat alur program sulit dilacak
- **Hindari BREAK/CONTINUE berlebihan** - Dapat mengurangi readability
- **Pertimbangkan alternatif** - Terkadang kondisi loop yang tepat lebih baik daripada BREAK
- **Gunakan CONTINUE untuk skip kondisi** - Lebih jelas daripada nested if
- **Gunakan RETURN untuk early exit** - Efisien untuk validasi input di awal method
- **Hindari multiple return points** - Kecuali untuk early validation yang jelas

Contoh Kasus

Studi Kasus 1: Validasi Input

Validasi Input dengan RETURN

```java
import java.util.Scanner;

public class InputValidator {
    public static void main(String[] args) {
        System.out.println("Masukkan usia:");
        Scanner input = new Scanner(System.in);
        int age = input.nextInt();

        if(age < 0) {
            System.out.println("Usia tidak valid");
            return;  // Hentikan program
        }
        if(age > 150) {
            System.out.println("Usia tidak realistis");
            return;  // Hentikan program
        }

        System.out.println("Usia valid: " + age);
    }
}

```

Studi Kasus 2: Pencarian Karakter

Pencarian dengan BREAK

```
1  public class CariKarakter {
2      public static void main(String[] args) {
3          String teks = "programming";
4          char target = 'g';
5
6          for(int i = 0; i < teks.length(); i++) {
7              if(teks.charAt(i) == target) {
8                  System.out.println("Karakter '" + target + "' ditemukan di index: " + i);
9                  break;  // Berhenti setelah ketemu pertama
10             }
11         }
12     }
13 }
14
```

Latihan

Latihan 1: Penghentian Pengecekan Faktor Bilangan Prima

Buat program yang menerima input sebuah bilangan bulat positif `n`. Program diminta menentukan apakah `n` hanya memiliki dua faktor positif, yaitu 1 dan `n` sendiri.

Gunakan **BREAK** untuk menghentikan proses pengecekan begitu ditemukan satu pembagi selain 1 dan `n`, agar program tidak melakukan pengecekan yang tidak perlu.

Contoh:

- Input: 13 $\rightarrow$ Output: `"13 termasuk bilangan prima"`
- Input: 21 $\rightarrow$ Output: `"21 bukan bilangan prima"`

Contoh Output

```
Masukkan bilangan:  13

13 termasuk bilangan prima
```

Latihan 2: Menampilkan Bilangan dengan Melewati Bilangan Genap

Buat program yang meminta input bilangan bulat `n`, lalu menampilkan semua bilangan bulat positif ganjil mulai dari 1 sampai `n`.

Gunakan **CONTINUE** untuk melewati bilangan genap di dalam perulangan, sehingga hanya bilangan ganjil yang ditampilkan.

Contoh:

- Input: `n = 10` → Output: `1 3 5 7 9`

Contoh Output

```
Masukkan bilangan bulat:  10

1 3 5 7 9
```

Kesimpulan

Kesimpulan

Inti Bab 8: Pernyataan Percabangan

- **BREAK**: Menghentikan perulangan/struktur kontrol secara paksa
- **CONTINUE**: Melompati iterasi saat ini dan lanjut ke iterasi berikutnya
- **RETURN**: Keluar dari method untuk method main. Untuk method non main bisa dengan nilai kembalian

Referensi

Referensi

Referensi:

- **Modul Praktikum Algoritma dan Pemrograman - Modul 8**
  Departemen Matematika FSAD ITS
- **Oracle Java Tutorials - Branching Statements**
  `https://docs.oracle.com/javase/tutorial/java/nutsandbolts/branch.html`
- **GeeksforGeeks - Java Break and Continue**
  `https://www.geeksforgeeks.org/break-and-continue-statement-in-java/`
- **GeeksforGeeks - Java Break Statement**
  `https://www.geeksforgeeks.org/java/break-statement-in-java/`
- **GeeksforGeeks - Java Continue Statement**
  `https://www.geeksforgeeks.org/java/continue-statement-in-java/`

Terima Kasih